

Tabla 1.1: Conjunto de registros de propósito general en RISC-V. La primera columna corresponde al nombre real de cada registro, la segunda al *alias* que puede usarse para referirse a cada uno de ellos —estos alias vienen definidos en la ABI, un aspecto avanzado que conoceremos en otro capítulo— y en la tercera se facilita una descripción breve de su finalidad.

La denominación *registro preservado* significa que el contenido de ese registro no será modificado al realizar una llamada a una función, por lo que su valor se preserva.

Un *frame* es una estructura de datos que se crea habitualmente al llamar a una función, con un espacio reservado para almacenar parámetros y otra información útil como la dirección de retorno.

Nombre	Alias	Uso
x0	zero	Siempre contiene el valor 0
x1	ra	Dirección de retorno
x2	sp	Puntero de pila
x3	gp	Puntero global
x4	tp	Puntero de hilo
x5 a x7	t0 a t2	Valor temporal
x8	s0/fp	Registro preservado / Puntero de <i>frame</i>
x9	s1	Registro preservado
x10, x11	a0, a1	Parámetro de función / Valor de retorno
x12 a x17	a2 a a7	Parámetro de función
x18 a x27	s2 a s11	Registro preservado
x28 a x31	t3 a t6	Valor temporal

1.2.2. Conjunto de instrucciones

[5]: Waterman et al. (2017), *The RISC-V Instruction Set Manual, Volume I: User-Level ISA*

3: En comparación, las últimas generaciones de microprocesadores x64 (familias Intel Core y AMD Ryzen) cuentan con un conjunto de unas 1500 instrucciones.

Además de su banco de registros, para escribir programas necesitaremos conocer también las instrucciones con que cuenta RISC-V [5]. Este conjunto cambia según las extensiones de la ISA que implemente el procesador. Para RV32I es bastante reducido³ al constar de **solo 40 instrucciones** distintas, con variantes según de dónde se tomen los operandos o el tipo de los mismos. A estas, RV32IM añade varias más para el cálculo de multiplicaciones y divisiones.

La [Tabla 1.2](#) resume el conjunto completo de instrucciones dividiéndolo en varios grupos y según que ambos operandos estén en registros o uno de ellos sea un valor inmediato. Conoceremos, en esta y las prácticas posteriores, cómo usar muchas de estas instrucciones.

Tabla 1.2: Conjunto de instrucciones de la ISA RV32IM.

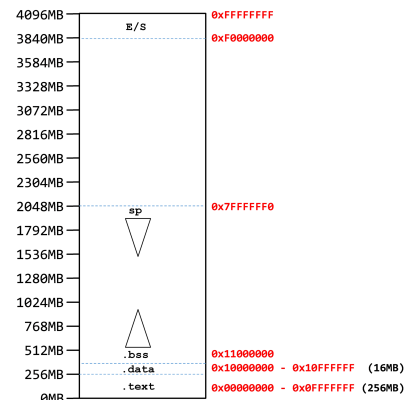
Grupo ↓	Tipo →	Registro	Inmediato
Aritmético		add, sub, mul, div, rem	addi, lui, auipc
Lógico		and, or, xor	andi, ori, xori
Desplazamiento		sll, srl, sra	slli, srli, srai
Comparación		slt, sltu	slti, sltiu
Carga			lb, lbu, lh, lhu, lw, lwu, ld
Almacenamiento			sb, sh, sw, sd
Salto			beq, bne, bge, bgeu, blt, bltu, jal, jalr
Otras		ecall, ebreak	

1.2.3. Seudoinstrucciones

4: La carga de un dato desde memoria a un registro, un cálculo aritmético sobre dos datos que están en registros, el almacenamiento del valor contenido en un registro en una determinada posición de memoria, etc.

La filosofía de los procesadores RISC, en términos generales, se basa en una ISA con pocas instrucciones encargadas de facilitar operaciones básicas⁴. Esto hace que la implementación de dicha ISA, en un microprocesador, sea relativamente sencilla, al tiempo que también se simplifican técnicas que conoceremos más adelante como la segmentación del cauce de ejecución.

Figura 1.7: Mapa de memoria del emulador Ripes, con las distintas secciones en que se divide y sus tamaños.



10: Trataremos este tema más adelante, en el capítulo 10. Operaciones de entrada y salida.

11: Esta generará dos instrucciones reales, las puedes observar en el panel derecho de la vista Editor de Ripes, que analizaremos después.

Las instrucciones **lw** y **sw** operan sobre números enteros de 32 bits. Existen las variantes **lb**, **lh** y **ld** para cargar números de 8, 16 y 64 bits, con sus correspondientes **sb**, **sh** y **sd** para almacenamiento.

CARGAR DOS NÚMEROS, SUMARLOS Y ALMACENAR EL RESULTADO

Listado 1.2: Leer dos números de memoria y escribir su suma.

```

1 | .data # Inicio del segmento de datos
2 | n1: .word 7 # Almacenamos en memoria
3 | n2: .word 3 # dos valores enteros
4 |
5 | .bss # Inicio de datos sin inicializar
6 | res: .word 0
7 |
8 | .text # Inicio del segmento de código
9 | lw a1, n1 # a1 = (n1)
10 | lw a2, n2 # a2 = (n2)
11 | add a0, a1, a2 # a0 = a1 + a2
12 | la t0, res # t0 -> res
13 | sw a0, 0, t0 # (res) = a0
14 | addi a7, zero, 10
15 | ecall # Llamar a la función 10 (exit)

```

12: Directivas análogas para otros tipos de datos son **.byte**, **.half** y **.long**, entre otras.

La sección **.text** contendrá el código del programa. En las secciones **.data** y **.bss** se alojan los datos inicializados y sin inicializar, respectivamente. El registro **sp** apunta a la parte superior de la pila y, a medida que se alojan datos en esta, crece hacia abajo. La zona marcada como **E/S** es el espacio reservado para la entrada/salida mapeada¹⁰.

1.5.2. Leer y escribir datos en memoria

La lectura de un dato que está almacenado en memoria conlleva asignar la dirección de memoria a un registro y, a continuación, usarlo como puntero para leer el dato en sí mismo. Es una operación que se simplifica gracias a la seudoinstrucción¹¹ **lw rd**, símbolo, en la que **rd** sería el registro de destino (donde se guardará el valor) y símbolo la etiqueta dispuesta delante de dicho dato al declararlo.

La instrucción complementaria a **lw** es **sw**. Su finalidad es almacenar el contenido de un registro en la dirección de memoria que se especifique. A diferencia de **lw**, sin embargo, **sw** no es una seudoinstrucción sino una instrucción real. Para usarla emplearemos la sintaxis **sw ro, desp, rdir**, en la que **ro** es el registro cuyo valor quiere escribirse en memoria, **rdir** el registro que actuará como puntero y **desp** un desplazamiento a sumar a la dirección contenida en **rdir**.

Antes de poder usar **sw**, por tanto, será preciso asignar al registro **rdir** la dirección de memoria que interese. Con este fin se usa la seudoinstrucción **la rdir**, símbolo. Como otras seudoinstrucciones, esta producirá dos instrucciones reales que aparecerán en el panel derecho. Serán estas las que obtengan la dirección que corresponda a símbolo y la almacene en el registro **rdir**.

Sobre la base de las instrucciones **lw**, **sw** y **la**, vamos a escribir un programa que cargue dos valores de una zona de memoria inicializada, los sume y almacene el resultado en una zona de memoria sin inicializar.

Las directivas **.data**, **.bss** y **.text** establecen el punto en el que comienzan las secciones de datos inicializados, datos sin inicializar y código, respectivamente. Cada una de estas secciones o segmentos tienen una dirección de inicio predeterminada. La directiva¹² **.word** reserva espacio para un entero de 32 bits.

La denominación asignada a cada parte de la instrucción es la usada en la especificación oficial de RISC-V.

1.6.2. Instrucciones tipo R

Estas son instrucciones en las que todos los valores que actúan como operandos se encuentran alojados en registros, de ahí que se vincule con el denominado **direccionamiento por registro**. El formato de la instrucción, con los 32 bits de la misma divididos en varios grupos, es el mostrado en la [Figura 1.9](#).

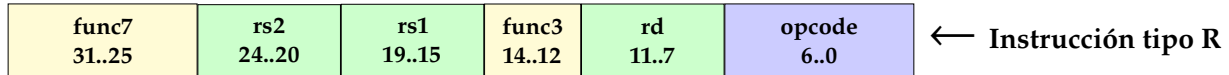


Figura 1.9: Formato de las instrucciones tipo R.

En el panel derecho de la vista **EDITOR** de Ripes, donde aparece el código máquina, puedes activar la opción **BINARY** para observar dicho código en formato binario, lo cual facilita la observación de las distintas partes de la instrucción.

Los tres registros implicados, **rs1**, **rs2** y **rd**, aportan los operandos sobre los que se actuará y el destino en el que se almacenará el resultado, respectivamente. A cada uno le corresponden 5 bits ($2^5 = 32$) ya que hay un total de 32 registros. En todos los formatos de instrucción los registros siempre ocupan las mismas posiciones, lo cual facilita la decodificación por parte del microprocesador.

El código de operación, conocido como *opcode*, siempre ocupa los siete bits de menor peso de la instrucción. Hay ciertos grupos de instrucciones que tienen el mismo *opcode*, caso en el que la operación que realizan viene determinada por un código de función de 10 bits. Estos son los que ocupan las posiciones **func3** y **func7**.

Un ejemplo de instrucción tipo R, que por tanto emplea direccionamiento por registro, es **add a0, a1, a2**, así como cualquier otra de las operaciones aritméticas y lógicas. Las instrucciones **add**, **sub**, **and**, **or** y **xor**, entre otras, tienen asociado un mismo *opcode*, siendo la unión **func7+func3** la que determina la operación a ejecutar.

1.6.3. Instrucciones tipo I y U

14: Nos referimos así a los valores literales o constantes que aparecen en la propia instrucción.

Este conjunto de instrucciones está asociado al **direccionamiento inmediato**¹⁴, caracterizado porque uno de los operandos es un valor inmediato en lugar de un registro, tal y como se aprecia en la [Figura 1.10](#) y la [Figura 1.11](#).

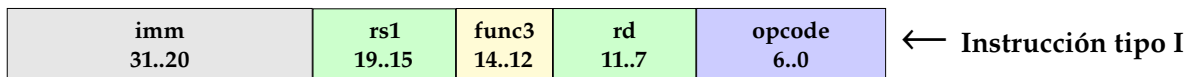


Figura 1.10: Formato de las instrucciones tipo I.

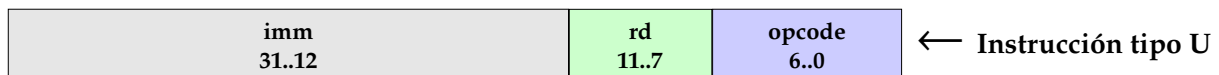


Figura 1.11: Formato de las instrucciones tipo U.

En el tipo I el valor inmediato ocupa los cinco bits que en el tipo R corresponden al **rs2**, más los 7 bits de **func7** por lo que, en total, se cuenta con 12 bits para el valor inmediato. Ya conocemos una instrucción de este tipo: **addi a0, a1, 5**, en la que **a0** es el registro de destino (**rd**), **a1** el primer operando (**rs1**) y el valor literal **5** el segundo (**imm**). Hay varias más similares, todas ellas compartiendo un mismo *opcode*, como son **andi**, **ori** o **xori**.

Hay que tener presente que, en general, RISC-V procesa los valores inmediatos como números enteros con signo. El bit de mayor peso denota si el número es positivo o negativo. Esto nos deja 11 bits para la magnitud (el valor absoluto), lo cual sería insuficiente para representar cantidades mayores que 2^{11} . En la práctica, esto implica que el rango de valores posibles sería estaría en rango $[-2048, 2047]$.

También son instrucciones de tipo I aquellas que emplean el **direccionamiento indexado**, consistente en tomar el contenido de un registro y sumarle el desplazamiento indicado por el valor inmediato, para obtener así el contenido de la

En RISC-V los números negativos se almacenan en **complemento a 2**. Por tanto, para obtener el negativo de cualquier número primero se invierten todos sus bits y, después, se suma 1. Así, y considerando que tenemos 12 bits para representar el valor, el número 5 (**0000 0000 0101**) pasaría a ser -5 tras invertir todos los bits (**1111 1111 1010**) y a continuación sumar 1 (**1111 1111 1011**).

dirección de memoria resultante. Es el caso de la instrucción **lw** *a0*, 128, *t0*, en la que al contenido del registro **t0** se le suma el desplazamiento 128 y, a continuación, se lee de [*t0*+128] una palabra para llevarla a **a0**.

El tipo U se caracteriza porque el valor inmediato tiene una longitud de 20 bits, en lugar de 12, y porque el valor se interpreta como un entero sin signo. Además, solo se cuenta con un registro como operando (*rd*) que actúa como destino de la operación. Una instrucción de este tipo es **lui** *t0*, *valor*, en la que los 20 bits de *valor* se transfieren como los 20 bits de mayor peso¹⁵ del registro **t0**, mientras que los 12 bits de menor peso se ponen a 0.

La otra instrucción que emplea el formato U es **auipc** *rd*, *imm*. Esta permite usar un tipo de **direccionamiento relativo** en el que uno de los operandos es fijo: el contador de programa (registro **pc**). Es un tipo de direccionamiento que permite a los compiladores producir código que puede cargarse en cualquier posición de la memoria, ya que todas las referencias a datos y etiquetas son relativas al **pc**.

15: Esto equivale a desplazar 12 bits hacia la izquierda el valor *o*, lo que es lo mismo, multiplicarlo por 4096 ($2^{12} = 4096$).

1.6.4. Instrucciones tipo S

Vinculado al conocido como **direccionamiento relativo**, este conjunto está formado por ciertas instrucciones de salto condicional. Se cuenta con dos operandos de entrada, alojados en sendos registros, sobre los cuales se llevará a cabo una operación, según cuyo resultado se modificará el **pc** sumándole el valor inmediato de 12 bits que se obtiene al concatenar *imm1* e *imm2*.

imm1 31..25	rs2 24..20	rs1 19..15	func3 14..12	imm2 11..7	opcode 6..0	← Instrucción tipo S
-----------------------	----------------------	----------------------	------------------------	----------------------	-----------------------	----------------------

Figura 1.12: Formato de las instrucciones tipo S.

Una instrucción que emplea este direccionamiento es **beq** *a0*, *a2*, *imm*, que compara el contenido de los registros **a0** y **a2** y, si es igual, modifica el **pc** en la magnitud indicada por el valor inmediato. En consecuencia, se produce un *salto* en el flujo de ejecución del programa.

Conoceremos las instrucciones de salto en el capítulo siguiente.

1.7. Aspectos avanzados

La seudoinstrucción **la**, que hemos usado en ejercicios previos para almacenar en un registro una dirección de memoria, produce dos instrucciones reales. Lo mismo ocurre con la seudoinstrucción **lw** cuando se usa para cargar un dato, no así con la instrucción del mismo nombre que, como ya sabemos, toma un parámetro adicional. Esto no es óptimo en un programa que realice muchos accesos a memoria. Veamos la vía para optimizar el código en estos casos.

lw es una seudoinstrucción cuando se usa solo con dos argumentos: el registro de destino y la dirección del dato a cargar. La instrucción real **lw** precisa tres: el registro de destino, un desplazamiento y un registro con la dirección base. Puede escribirse como **lw** *rd*, *desp*, *rbase* o bien como **lw** *rd*, *desp*(*rbase*).

1.7.1. Asignación de una dirección a un registro

Ahora que conocemos la configuración de memoria con la que trabajaremos, así como los modos de direccionamiento existentes, lo que nos interesa es saber cómo podemos asignar a un registro una cierta dirección de memoria. Completada esa operación, el registro servirá como puntero para acceder a esa dirección y otras adyacentes mediante la suma/resta de un desplazamiento.

En el anterior [Listado 1.2](#) hemos usado la seudoinstrucción **lw** para cargar nuestros datos. Esta produce dos instrucciones reales. Si vamos a cargar muchos datos desde memoria, una de ellas será reiterativa, ya que la primera instrucción siempre cargará una dirección base en un registro, mientras que la segunda aplicará un desplazamiento sobre la misma.

El [Listado 1.3](#) muestra cómo usar las instrucciones **lui** y **addi** para cargar en el registro **t0** —que pretendemos usar como si fuese un puntero— la dirección de 32 bits que corresponde a la etiqueta *par*. Descomponemos la dirección de dicha